Rockchip RT-Thread SPI2APB 升级开发指南

文件标识：RK-KF-YF-F01

发布版本：V1.0.0

日期：2024-09-04

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文介绍了基于 RT-Thread 系统SPI2APB升级开发指南及相关配置。

各芯片系统支持状态

芯片名称	内核版本
RK2118	RT-Thread v4.1.x

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	Looper Zheng、Roger Hu	2024-09-04	初始版本

目录

Rockchip RT-Thread SPI2APB 升级开发指南

1. SPI2APB 升级介绍
   1.1 SPI2APB 升级优势：
2. 硬件配置
   2.1 硬件连接
   2.2 复位RK2118进入升级模式
3. 升级流程
   3.1 升级介绍
   3.2 具体升级流程
   3.3 流程图
   3.4 伪代码说明

1. SPI2APB 升级介绍

SPI2ABP升级指的是其它主控例如车载中的MCU芯片通过SPI接口对RK2118芯片进行固件升级。

SPI2APB在标准的SPI通讯协议上叠加RK私有协议，具体协议格式可以参考《Rockchip_Developer_Guide_RT-Thread_SPI2APB_CN》。

1.1 SPI2APB 升级优势：

- 支持空片升级
- 升级安全，不怕升级过程中断电等异常
- 节省flash空间，不需要A/B固件

2. 硬件配置

2.1 硬件连接

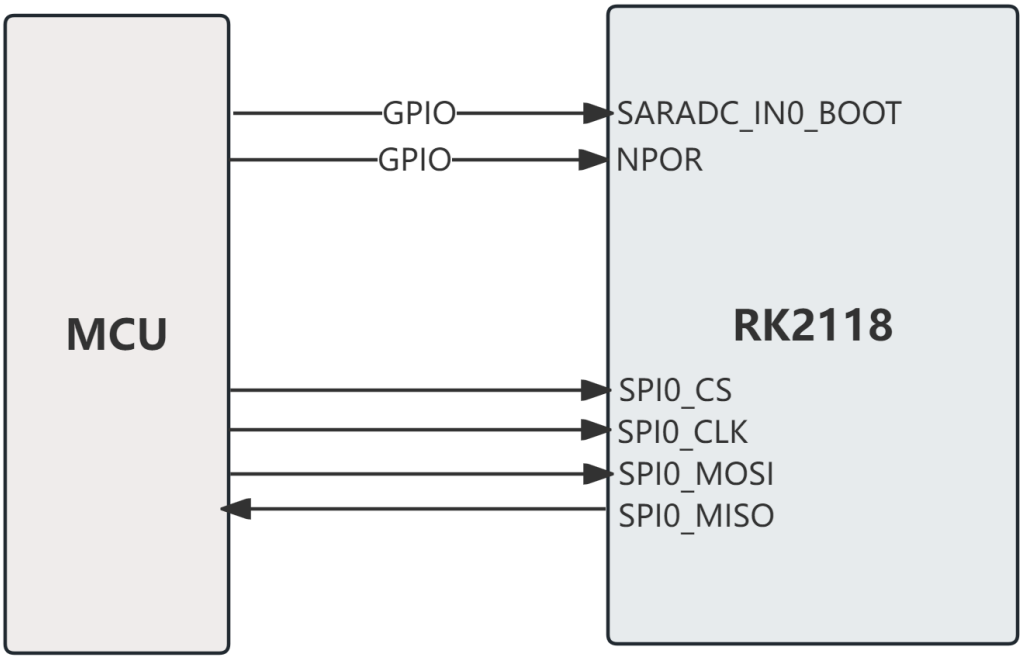
需要接RK2118的SPI0口，有M0或M1可以选择：

```
ETH_RMII_CRSDV | SPI0_CLK_M0     | RM2_IO0 | GPIO2_A0_d
ETH_RMII_MDIO  | SPI0_CSN_M0     | RM2_IO1 | GPIO2_A1_d
ETH_RMII_MDC   | SPI0_MOSI_M0    | RM2_IO2 | GPIO2_A2_d

ETH_RMII_TXEN  | SPI0_MISO_M0    | RM2_IO3 | GPIO2_A3_d

 EMMC_D5   | FSPI_D5    | SPI0_CLK_M1   | GPIO1_A0_u
 EMMC_D3   | FSPI_D7    | SPI0_CSN_M1   | GPIO1_A1_u
 EMMC_D4   | FSPI_D6    | SPI0_MOSI_M1  | GPIO1_A2_u
 EMMC_D0   | FSPI_D4    | SPI0_MISO_M1  | GPIO1_A3_u
```

SPI0硬件连接如下：

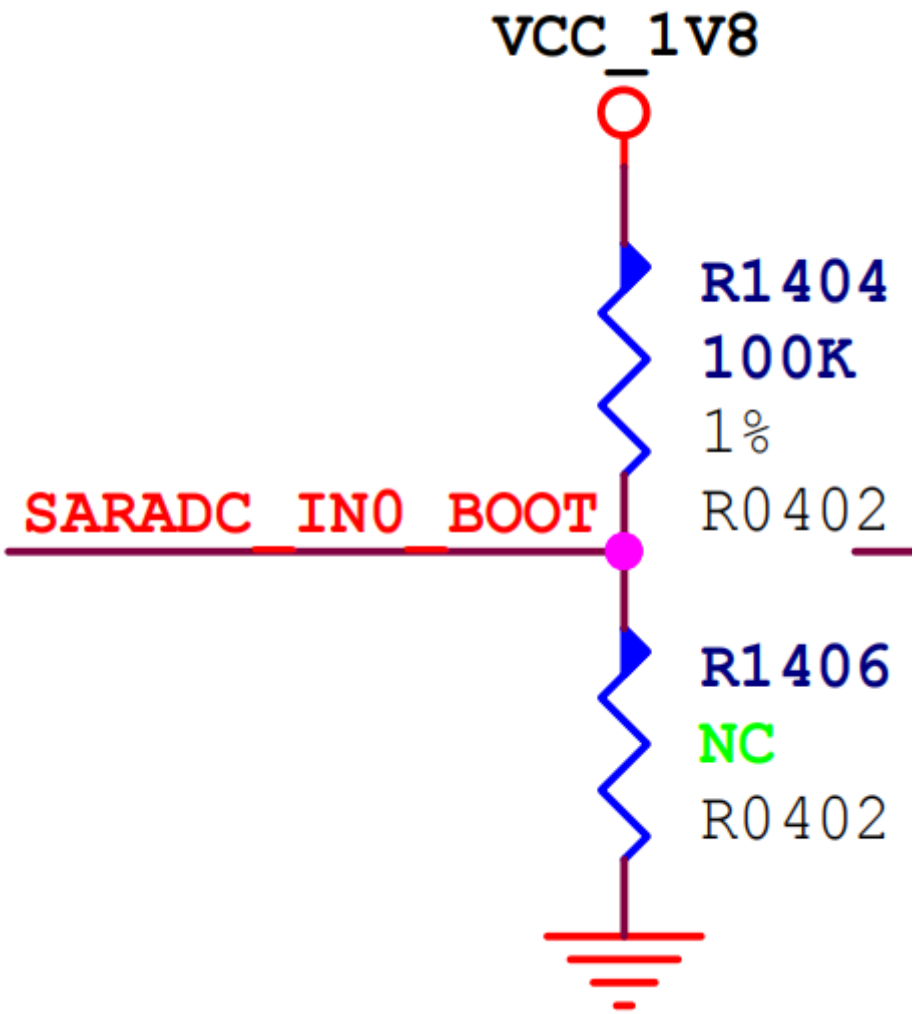


1. 两个GPIO用于控制RK2118进入Maskroom
2. SPI0用于升级和正常通信
3. spi2apb升级方式优点：可靠、节省flash空间不需要A/B固件）

2.2 复位RK2118进入升级模式

控制NPOR与SARADC_IN0_BOOT脚来让RK2118进入升级模式(maskroom)。

SARADC_IN0_BOOT控制RK2118启动模式，根据硬件接的是M0还是M1来配置分压电阻，具体如下：



BOOT MODE:

Ru	Rd	ADC	BOOT MODE
NC	100K	0	USB/UATR0/SPI0_M0
100K	12K	110	USB/UATR0/SPI0_M1
100K	27K	217	FSPI(1x/4x)->FSPI(8x)->EMMC->SDMMC->USB/UATR0/SPI0_M0
100K	51K	346	FSPI(1x/4x)->FSPI(8x)->EMMC->SDMMC->USB/UATR0/SPI0_M1
100K	82K	461	FSPI(8x)>USB/UATR0/SPI0_M0
100K	120K	558	
100K	200K	682	EMMC->USB/UATR0/SPI0_M0
100K	330K	785	
100K	820K	912	FSPI(1x/4x)->USB/UATR0/SPI0_M0
100K	**NC**	**1023**	**FSPI(1x/4x)->USB/UATR0/SPI0_M1**

NPOR脚用来复位RK2118。

这两个IO具体控制时序为：同时拉底SARADC_IN0_BOOT与NPOR，等1ms后再拉高保持，具体可见下一节的spi_entry_maskroom。

3. 升级流程

3.1 升级介绍

需要先升级引导程序rk2118_upgrade.bin，再升级正式的固件Firmware.img。

可用以下工具将两个固件打包成一个rk2118_spi2apb_update.img。

```
cd bsp/rockchip/rk2118
./package_spi2apb_image.py rkbin/rk2118_upgrade.bin Image/Firmware.img

生成
Image/rk2118_spi2apb_update.img
```

SPI2APB升级参考dmeo：

```
bsp/rockchip/common/tests/spi2apb_upgrade_test.c
```

它从rk2118_spi2apb_update.img中提取出rk2118_upgrade.bin与Firmware.img进行升级。

SPI2APB控制地址：

简介	定义	地址
IMEM_BASE_ADDR	基地址	0x00200000
SPI_0471_ADDR	引导程序下载地址	(IMEM_BASE_ADDR + 4*1024)
SPI_XFER_ADDR1	固件SRAM暂存地址	(IMEM_BASE_ADDR + 64*1024)
SPI_REQ_ADDR	指令请求地址	(IMEM_BASE_ADDR + 1024*1024 - 64)
APB_CMD_WRITE	写数据指令	0x00000011
APB_CMD_READ	读数据指令	0x00000077
APB_CMD_QUERY_REG2	查询内部 REG2 寄存器，用于查询内部状态机	0x000001FF

3.2 具体升级流程

主机指MCU，用来给RK2118 DSP升级；从机指RK2118 DSP。

1. 控制**RK2118**进入**maskroom**升级模式

1.1 控制 NPOR 和 SARADC 引脚， 让从机进入maskroom

RK2118芯片中固化程序(maskroom)会通过SARADC_IN0_BOOT脚的电平来确定用哪一组SPI2APB 引脚来进行通信（M0/M1）

1.2 读取从机状态，确定从机是否已进入maskroom

读取 SPI2APB QUERY REG2 (0x000001FF) 寄存器状态， 判断是否已经进入了maskroom状态(0xAA55)

2. 烧录引导程序 **rkbin/rk2118_upgrade.bin**

2.1 烧录引导程序

主机向从机SRAM （SPI_0471_ADDR）地址写入引导程序

2.2 启动引导程序

主机向从机（SPI_REQ_ADDR）发送启动引导程序指令(0x0471)

2.3 读取从机状态， 确定引导程序是否启动正常

读取 SPI2APB QUERY REG2 寄存器状态， 判断是否已经启动引导程序(0x0471)

3. 烧录固件 **Firmware.img**

3.1 主机向从机SRAM SPI_XFER_ADDR1 地址分块写入固件数据

3.2 主机向从机发送flash写入请求指令，从SPI_XFER_ADDR1地址写到flash中特定地址

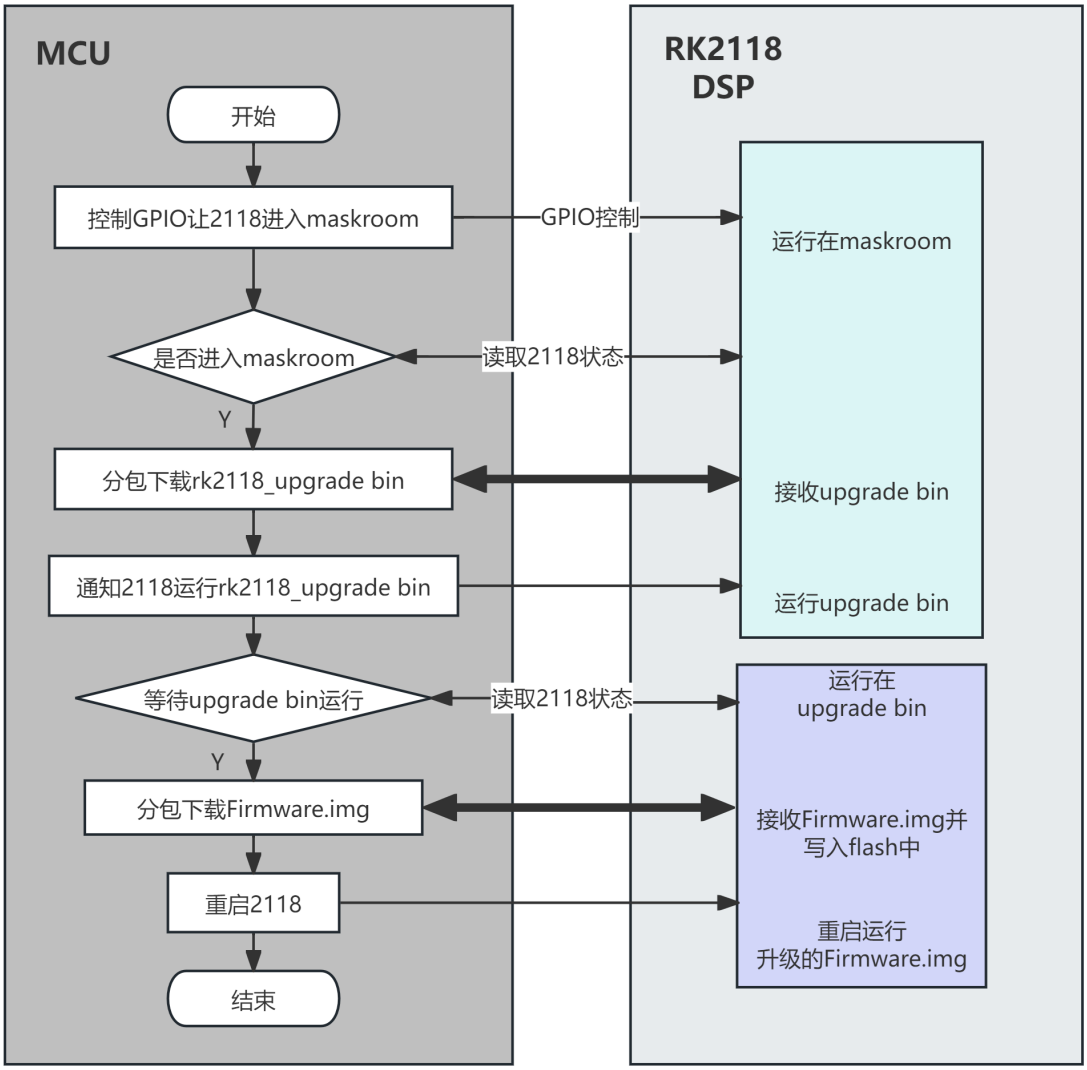
3.3 主机查询 REG2 寄存器状态， 确定固件校验及写flash动作已经执行结束

3.4 回到3.1，直接整个固件写入完成

4. 重启平台

4.1 主机向从机(SPI_REQ_ADDR)地址发送重启平台指令

3.3 流程图



3.4 伪代码说明

```
# 控制RK2118进入升级模式
rt_int32_t spi_entry_maskroom()
{
    HAL_GPIO_SetPinsLevel(GPIO2, GPIO_BANK1, GPIO_HIGH);     // BOOT PIN
    HAL_GPIO_SetPinsLevel(GPIO2, GPIO_BANK2,GPIO_LOW);       // RESET PIN

    rt_thread_delay(RT_TICK_PER_SECOND / 1000);

    HAL_GPIO_SetPinsLevel(GPIO2, GPIO_BANK2 ,GPIO_HIGHT);    // RESET PIN
    HAL_GPIO_SetPinsLevel(GPIO2, GPIO_BANK1 ,GPIO_LOW);      // BOOT PIN
    return RT_EOK;
}

# 等待RK2118进入特定启动状态
rt_int32_t wait_boot_ready(rt_uint32_t flag)
{
    do{
        status = spi_ready_status();
        if ((status & 0xFFFF) == flag)
            return RT_EOK;
```

```
        rt_thread_delay(RT_TICK_PER_SECOND / 1000);
    }while(timeout--);

    return -RT_ERROR;
}

# 下载引导程序
rt_int32_t spi_down_bin(rt_uint16_t cmd, rt_uint32_t load_addr, rt_int32_t fd)
{
    while(1)
    {
        // read upgrade bin
        readed = read(fd, write_buf, SPIDEV_RKSPI2APB_OP_BYTES);

        // spi transfer bin data
        ret = spi_write_io(src_addr, (void *)write_buf, readed);

        src_addr += readed; total += readed;
    }

    // request boot upgrade program
    ret = spi_send_request(cmd, load_addr, total, (void *)NULL, BOOT_FLAG_CRC |
BOOT_FLAG_RUN);
    return RT_EOK;
}

# 下载固件Firmware.img
rt_int32_t spi_down_img(rt_uint16_t cmd, rt_uint32_t load_addr, rt_int32_t fd)
{
    while (1)
    {
        // read frameware image
        readed = read(fd, write_buf, SPIDEV_RKSPI2APB_OP_BYTES);

        // spi frameware image data
        ret = spi_write_io(SPI_XFER_ADDR1, (void *)write_buf, readed);

        // reqeuest wirten from sram to flash
        req.buf_addr = SPI_XFER_ADDR1;
        req.lba = addr;
        req.nsec = nsec;
        ret = spi_write_io(SPI_REQ_ADDR, (void *)&req, sizeof(spi_request_t));

        // query request complete!
        ret = wait_boot_ready(0x0000);
        sec_addr += nsec; total += readed;
    }

    return RT_EOK;
}

# 重启RK2118
rt_int32_t spi_reset_boot(void)
{
    // request reboot platformat
    req.cmd = SPI_FW_RESET;
    ret = spi_write_io(SPI_REQ_ADDR, (void *)&req, sizeof(spi_request_t));
    return RT_EOK;
```

```
}

# 整个升级流程
rt_int32_t spi_upgrade(har *bin_name, char *img_name)
{
    file_bin = open((const char *)bin_name, O_RDONLY, 0);
    file_img = open((const char *)img_name, O_RDONLY, 0);

    // entry maskroom
    spi_entry_maskroom();

    // ready reg2 status
    ret = wait_boot_ready(0xAA55);

    // download upgrade bin
    ret = spi_down_bin(0x0471, SPI_0471_ADDR, file_bin);

    // ready reg2 status
    ret = wait_boot_ready(0x0471);

    // donwload frameware image
    ret = spi_down_img(0, 0, file_img);

    // reset platform
    spi_reset_boot();

    return RT_EOK;
}
```